

Contents

Voxel-Steered Autoregressive Generation: Overcoming the Quadratic Output Wall	1
Abstract	1
1. The Asymmetry of Autoregression	2
2. Voxel Memory for Generative Output	2
3. Greedy Meshing for Output Deduplication	3
4. Architecture Implementation (The Rolling Loop)	3
5. What is measured	4
6. Claims not made	6
7. Limitations	6
8. Conclusion	7
Related work in this series	7

Voxel-Steered Autoregressive Generation: Overcoming the Quadratic Output Wall

Authors: Weslyn Cory Whitehead Jr.¹

Affiliations:

¹ AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: <https://orcid.org/0009-0005-7707-3210>

Submitted: June 2026

Working Paper Series: AAMT-WP-21

Anchor DOI: 10.5281/zenodo.19600795

License: Creative Commons Attribution 4.0 International (CC BY 4.0)

Status: Working paper, not peer reviewed. The Rolling Output Eviction loop and TERA-based output steering algorithms are implemented within the MaiaM Alchemist engine (apps/desktop/scripts/wp21_rolling_output_prototype.py, embedder_bridge.py, memory_index_bridge.py). This paper extends **WP-20 (Voxel-Addressable Memory)** by inverting the retrieval mechanism from the input context space to the generative output space, so that addressable output history is unbounded while the working set stays fixed. §5 states which quantities the prototype measures, which it does not, and the experiment that would falsify the central claim; §6 and §7 state what is not claimed and where the argument is weakest. Revision 2 (August 2026) exists because the first version asserted “infinite-length coherent generation” without measurement — see §6.

Abstract

Autoregressive language models face a strict quadratic wall: generation costs $O(N^2)$ in attention and $O(N)$ in Key-Value (KV) cache memory. To generate coherent long-form text (e.g., entire books), current models must hold the entire generated history in a dense KV cache. We present

Voxel-Steered Autoregressive Generation, a geometric inversion of WP-20’s spatial index. By implementing a “Rolling Output Eviction” loop, the model evicts recently generated chunks from the dense KV cache into a persistent HNSW index over the TERA cube, querying its own past geometry in an attempt to carry coherence forward. Furthermore, by predicting coarse semantic cells (Meji/Odu) before decoding discrete tokens, the model generates a trajectory through meaning-space rather than a sequence of characters. The architecture is designed so that KV-cache memory is $O(k)$ in the window rather than $O(N)$ in the output length, which makes total generation cost linear in N rather than quadratic. Whether the retrieved geometry preserves coherence over long outputs is the open question: the prototype instruments it with WP-22’s coverage metrics, and §5.3 states the A/B that would settle it. This paper describes a mechanism and reports what the prototype measures; it does not claim a coherence result.

1. The Asymmetry of Autoregression

The context limit of a Transformer applies identically to input prompts and generated outputs, but generation exacerbates the hardware penalty. To predict token $N + 1$, the model must attend to all N preceding tokens. Generating a 100,000-word novel token-by-token inflates the dense KV cache until the hardware runs out of memory.

Information theory indicates that retaining exact sequences of tokens is wildly inefficient for maintaining narrative or logical coherence. A human writing a novel does not hold the exact wording of chapter one in their working memory while writing chapter ten; they hold the *geometric attractor* (the plot, the archetype, the tone) and retrieve specifics on demand.

To break the autoregressive bottleneck, we must stop holding tokens and start indexing geometry.

2. Voxel Memory for Generative Output

WP-20 introduced a coarse-to-fine hierarchy over the unit TERA (Temporal, Emotional, Rational, Archetypal) cube: 1. **Meji (L1)**: 4-bit, 16 cells (coarse intent). 2. **Odu (L2)**: 8-bit, 256 cells (fine intent). 3. **Residual (L3)**: Whitened sentence embedding indexed via HNSW.

In WP-21, we apply this identical spatial index to the **output stream**.

2.1 Rolling Output Eviction (Tier 0 to Tier 1)

Instead of a monolithic KV cache that grows linearly with generation, we define: - **Tier 0 (Working Set)**: A strict sliding window of the last k generated tokens (e.g., $k = 512$). - **Tier 1 (Geometric Store)**: The HNSW index of all previously generated text chunks.

When a semantic chunk (a sentence or paragraph) passes out of the Tier 0 window, it is not discarded. It is embedded, whitened, and stored in the Tier 1 Voxel space.

As the model generates new text, the current thought acts as a query to the Voxel space. The engine retrieves the k -nearest geometrically similar past thoughts and injects them into the prompt. The *addressable* history therefore grows without bound while the active KV cache stays bounded at $O(k)$. Two distinctions matter and were elided in the first version of this paper. The addressable history is unbounded; the *effective* context at any step is only what retrieval surfaces — here $k = 3$

chunks — which is a small window onto that history, not the whole of it. And bounded is not $O(1)$: attending over a fixed window costs $O(k)$ per token, so generation is linear in N rather than quadratic. Linear-not-quadratic is the real claim and it is strong enough without overstating it.

2.2 Hierarchical Generation (Coarse-to-Fine Decoding)

Standard Transformers predict discrete tokens directly from a vast vocabulary space ($\approx 50,000$ tokens). Voxel-steered generation separates semantic planning from token decoding:

1. **Predict Meji:** The model predicts the next 4-bit Meji coordinate, determining the structural intent of the next paragraph.
2. **Predict Odu:** The model refines the 4-bit coordinate to an 8-bit Odu target.
3. **Decode Tokens:** A local decoder conditioned on the Odu spatial coordinate fills in the discrete tokens.

The generation becomes a trajectory over a 4D manifold. We are no longer generating tokens; we are steering through meaning-space.

3. Greedy Meshing for Output Deduplication

A major inefficiency in long-form generation (especially code generation) is the repetition of identical semantic structures. If the model generates standard boilerplate or repeated logical arguments, it wastes compute recalculating the attention matrices for identical concepts.

Applying the **Greedy Meshing** principle from 3D voxel rendering to the generative output: - When the model's intended trajectory nears a Tier 1 attractor that is highly similar ($\text{cosine} > 0.95$) to a previously generated chunk, it does not decode the tokens. - Instead, it outputs the **sparse chunk address** (the Morton key) of that geometric cell. - The runtime engine interpolates the cached text into the output stream directly.

This Hopfield-style consolidation is intended to suppress verbatim self-repetition and strip redundant compute. It is a substitution, not a guarantee: above the threshold the runtime emits cached text in place of decoded text, so the output is no longer a sample from the model, and a chunk that is *near* a past chunk but meaningfully different is exactly the case the threshold gets wrong. §5.2 reports the substitution count; §7 states why the count alone does not establish that the substitutions were correct.

4. Architecture Implementation (The Rolling Loop)

The core loop implemented in the MaiaM engine (`wp21_rolling_output_prototype.py`, 683 lines) is sketched below. The listing is the shape of the loop, not the harness — the running program additionally drives the two bridge processes, computes the WP-22 coverage metrics reported in §5.2, and performs the greedy-mesh check after generation rather than before:

```
# The Rolling Output Eviction Loop
while True:
    # 1. Query Voxel Space for relevant past thoughts
```

```

past_thoughts = memory_index.query(current_thought_embedding, k=3)

# 2. Inject into Tier 0 Working Set
prompt = build_prompt(past_thoughts, recent_context)

# 3. Generate Next Chunk (Tier 0)
generated_chunk = llm.generate(prompt)

# 4. Evict to Tier 1
chunk_embedding = embedder.embed(generated_chunk)
memory_index.add(chunk_embedding, generated_chunk)

# 5. Slide Window
update_recent_context(generated_chunk)

```

Decoupling the retained state from the attention matrix removes the mechanism by which long generation exhausts memory: the prompt handed to the model is bounded by construction, so it cannot grow into an out-of-memory condition however long the output becomes. Coherence is a separate question and is not settled by the same argument — the loop guarantees a bounded prompt, not a good one. The prototype instruments coherence with the proxies in §5.2 rather than asserting it.

5. What is measured

The series convention is to separate measured from derived from pending, and to name the harness for anything pending rather than estimate it. The first version of this paper had no such section, which is how “infinite-length coherent generation” reached an abstract unchallenged.

5.1 Inherited, and binding

- **Pure-TERA retrieval recall@10 = 0.017** (WP-20 §4.2). This is the measured discrimination of coarse TERA routing on its own, and it bounds this paper the same way it bounds WP-26 §3.3. The Meji/Odu levels *route*; the L3 whitened- residual HNSW index is what retrieves. Any reading of §2.2 in which the coarse cells do the semantic work is a misreading of a measured result.

5.2 Measured by the prototype

wp21_rolling_output_prototype.py is instrumented with the WP-22 Odu Coverage Map (odu_coverage_map.py, OduCoverageMap.summary()) and emits, per session of N chunks:

Quantity	Reported as	Stated target	Looping baseline
Session entropy over Odu cells	h_session (bits)	> 4.0	≈ 0.3
Odu cell coverage	coverage_pct	> 30%	≈ 4%
Collapsed TERA axes	collapsed_axes	0–1	2–3
Greedy-mesh substitutions	meshes_compressed	—	—
Steering interventions	steers_triggered	—	—

Quantity	Reported as	Stated target	Looping baseline
Mean pairwise chunk cosine	avg_pairwise_cosine	< 0.60	> 0.85

These are the honest measurable stand-ins for the words the first version used directly. avg_pairwise_cosine and h_session are *repetition and scatter* proxies, not coherence; a session can score well on both and still be incoherent, because neither reads the text. They are reported because they are what the harness produces, and labelled as proxies because that is what they are.

5.3 Pending — the harness exists to produce these

#	Quantity	Harness	Falsifies
1	Wall-clock and peak RSS vs output length N	prototype, WP21_ITERATIONS sweep	§2.1's linear-not-quadratic claim
2	Human or LLM-judged coherence at $N = 10^3, 10^4, 10^5$ tokens, rolling loop vs full-context baseline on the same seeds	not yet built	the central claim
3	Greedy-mesh substitution correctness — how many of meshes_compressed were semantically right	manual audit of the substitution log	§3
4	Retrieval hit quality: fraction of retrieved chunks a reader judges relevant	prototype query_index_with_dist diagnostics	§2.1
5	Sensitivity of every result to the cosine 0.95 and $k = 3$ constants	parameter sweep	all of the above

Experiment 2 is the one that matters and it has not been built. Until it exists, the claim that this architecture maintains coherence over long outputs is a hypothesis with a mechanism attached, not a result.

5.4 What the prototype does not establish

The prototype generates through **OpenRouter** (`generate_openrouter`), a remote API. It therefore bounds the *prompt* it constructs; it does not own or measure a KV cache. The $O(k)$ memory argument in §2.1 follows from the prompt being bounded by construction and is sound as an architectural statement, but this harness is not evidence about any particular engine’s cache behaviour. A local run against a controlled decoder is what would turn the argument into a measurement, and it is experiment 1.

6. Claims not made

- **No coherence result.** The mechanism is specified and instrumented; §5.3 experiment 2 is unbuilt. “Infinitely long, deeply coherent” — the first version’s phrasing — asserted precisely the thing that was never measured.
 - **No $O(1)$ attention claim.** Attention over a fixed window is $O(k)$ per token, giving $O(Nk)$ overall. Linear rather than quadratic is the claim.
 - **Nothing is infinite.** Addressable history is unbounded; effective context is k retrieved chunks. Physical storage bounds the first and k bounds the second.
 - **No claim that coarse Meji/Odu prediction retrieves well.** WP-20 measured `recall@10` = 0.017 for exactly that. The hierarchy routes.
 - **No throughput or quality comparison against any long-context method** — not against sliding-window attention, not against Transformer-XL-style recurrence, not against a long-context model. None has been run.
 - **No claim that greedy meshing preserves meaning.** It substitutes cached text above a similarity threshold. §5.3 experiment 3 is how that would be checked.
-

7. Limitations

The central claim is unmeasured. Everything in §2 and §3 stands on the hypothesis that geometrically retrieved past chunks preserve coherence as well as a full context would. That comparison has not been run.

Greedy meshing changes the output distribution. Above cosine 0.95 the runtime emits cached text instead of decoded text. The output is then not a sample from the model, and the threshold is a free parameter chosen without a sweep. A model that is *about* to say something subtly different from a past chunk is the case this gets wrong, and it will get it wrong silently.

Retrieval inherits WP-20’s ceiling. Coarse routing has measured `recall@10` of 0.017. If the L3 residual index is weak on a given corpus, the loop retrieves plausible-but-wrong context and steers the generation into it — which reads as confident drift rather than as an error.

The eviction boundary is semantic, and the segmenter is not evaluated. Chunks are evicted at sentence or paragraph boundaries. A bad split puts half a thought in the index, and nothing here measures how often that happens.

Bounded prompt is not bounded cost end to end. Retrieval, embedding and index insertion are per-chunk costs the quadratic-attention framing ignores. At small N they may dominate the

saving; experiment 1 is what would show the crossover.

No adversarial or degenerate-input testing. A generation that legitimately needs to repeat itself — a table, a refrain, a formal proof restating a lemma — is precisely what greedy meshing will suppress.

8. Conclusion

Voxel-Steered Autoregressive Generation applies WP-20’s spatial index to the output stream rather than the input context. Evicting generated chunks into a geometric store and retrieving from it keeps the prompt bounded by construction, which changes generation cost from quadratic to linear in output length and removes the mechanism by which long generation exhausts memory. That much is a consequence of the construction and does not need an experiment.

Whether it preserves coherence is a different question, and this paper does not answer it. The first version of this paper said it did — “infinitely long, deeply coherent sequences” — on the strength of a mechanism and no measurement. This revision states the mechanism, reports the coverage proxies the prototype actually produces, and writes down the A/B against a full-context baseline that would settle it, before the result is known.

Related work in this series

- **WP-20** — Voxel-Addressable Memory: the coarse-to-fine hierarchy this paper inverts, and the $\text{recall@10} = 0.017$ measurement that bounds §5.1.
- **WP-22** — Odu Coverage Maps: the metrics §5.2 reports.
- **WP-23** — The .aamt Substrate: the production storage layer this prototype’s HNSW index stands in for.
- **WP-26** — The Experience Substrate: instantiates this paper’s steering primitive inside a decode loop, and states the same coherence question as its own experiment 4.